Knowledge-Based Systems 186 (2019) 104975

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Short communication

Unsupervised domain adaptation: A multi-task learning-based method☆

Jing Zhang, Wanqing Li *, Philip Ogunbona

School of Computing and Information Technology, University of Wollongong, Australia

ARTICLE INFO

Article history:
Received 6 December 2018
Received in revised form 29 July 2019
Accepted 19 August 2019
Available online 22 August 2019

Keywords:
Unsupervised domain adaptation
Transfer learning
Object recognition
Digit recognition

ABSTRACT

This paper presents a new perspective to formulate unsupervised domain adaptation as a multi-task learning problem. This formulation removes the commonly used assumption in the classifier-based adaptation approach that a shared classifier exists for the same task in different domains. Specifically, the source task is to learn a linear classifier from the labelled source data and the target task is to learn a linear transform to cluster the unlabelled target data such that the original target data are mapped to a lower dimensional subspace where the geometric structure is preserved. The two tasks are jointly learned by enforcing the target transformation is close to the source classifier and the class distribution shift between domains is reduced in the meantime. Two novel classifier-based adaptation algorithms are proposed upon the formulation using Regularized Least Squares and Support Vector Machines respectively, in which unshared classifiers between the source and target domains are assumed and jointly learned to effectively deal with large domain shift. Experiments on both synthetic and real-world cross domain recognition tasks have shown that the proposed methods outperform several state-of-the-art unsupervised domain adaptation methods.

© 2019 Elsevier B.V. All rights reserved.

1. Introduction

It is generally assumed that the training and test data are drawn from the same distribution in statistical learning theory. Unfortunately, this assumption does not hold in many applications. A well-studied strategy to address this issue is domain adaptation [1–5] which employs previous labelled source domain data to boost the task in a new target domain with a few or even no labelled data.

In this paper, we focus on the problem of unsupervised domain adaptation in which source data are labelled, but available data in the target domain are unlabelled. Specifically, we consider homogeneous domain adaptation, where the feature spaces between the source and target domains are the same, which is different from heterogeneous domain adaptation [6–8]. One approach to this problem is referred to as feature transformation-based domain adaptation [3,4,9–12], which transforms the original feature into another space where the distributions of the two domains would be similar. A classifier can then be trained in the space for both source and target data. Another approach [2,13], referred to as classifier-based domain adaptation, aims at learning a single and shared classifier directly such that the distribution of classification scores on the source data aligns with that on the target data. However, there may not exist such a classifier that could perform well on both domains, especially when the domain shift is large. An alternative approach is to jointly learn two classifiers, one is optimized for the source domain and the other is optimized for the target domain. Yang et al. [1] proposed a method to learn two classifiers for the source and target domain respectively for the case where both the source and target data are labelled (i.e. supervised domain adaptation). Duan et al. [14] propose a multi-source domain adaptation method that requires labelled target data. Recently, several deep-learning based domain adaptation methods are also proposed (DANN [15], DAN [16], RTN [17], and WDAN [18]). They extend deep CNNs to domain adaptation either by adding additional layers through which the mean embeddings of distributions are matched (DAN [16], RTN [17], and WDAN [18]), or by adding a sub-network as a domain discriminator whilst the deep features are learned to confuse the domain discriminator in a domain-adversarial training paradigm (DANN [15]). Since most of these deep learning methods still follow the idea of shared classifiers (parameters), their capability of adaptation would drop when the shift becomes large.

Unlike the works [2,13,16,18] that generally assume the existence of a shared classifier that could perform well on both source and target domain, and the works [1,14] that require the

☆ No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to https://doi.org/10.1016/j.knosys.2019.104975.

* Corresponding author.
E-mail addresses: jz960@uowmail.edu.au (J. Zhang), wanqing@uow.edu.au (W. Li), philipo@uow.edu.au (P. Ogunbona).

https://doi.org/10.1016/j.knosys.2019.104975
0950-7051/© 2019 Elsevier B.V. All rights reserved.

Table 1
Notations and descriptions.

Notation	Description	Notation	Description
$\mathbf{x}_i^s$, $\mathbf{x}_j^t$	features	γ_A	shrinkage regularization parameter
y_i, y_j	labels	γ_M	classifier regularization parameter
$f(x)$	function in $\mathcal{H}$	γ_I	manifold regularization parameter
$\mathbf{f}_s$, $\mathbf{f}_t$	decision values from classifiers	γ_D	MMD regularization parameter
K, K_s, K_t	kernel matrices	M	MMD matrix
n_s, n_t	# samples	L	graph Laplacian
$\boldsymbol{\alpha}_s$, $\boldsymbol{\alpha}_t$	coefficients	C	# classes

availability of labelled target data, this paper provides a new perspective that unsupervised domain adaptation can be formulated as a multi-task learning problem and proposes a novel classifier-based method to implement this formulation. Such a formulation allows us to learn unshared classifiers between the source and target to deal with the large shift. Specifically, the source task is to learn a linear classifier from the labelled source data and the target task is to learn a linear transform to cluster the unlabelled target data such that the original target data are mapped to a lower dimensional subspace where the geometric structure is preserved. To effectively transfer the discriminative information from the source domain, the two tasks are jointly learned by enforcing the target transformation is close to the source classifier and the class distribution shift between domains is reduced in the meantime, such that the mapped target data are clustered to their corresponding classes.

The proposed method is inspired by the adaptation regularization based transfer learning method (ARTL) proposed by Long et al. [13]. The ARTL method learns a shared adaptive classifier by simultaneously optimizing the structural risk functional, the joint distribution matching between domains, and the manifold consistency underlying marginal distribution. The ARTL is an effective domain adaptation method which learns the adaptive classifier directly without the need of intermediate subspace learning procedures, and the state-of-the-art results were obtained on many cross-domain recognition datasets at the time. However, there are two key differences between our method and ARTL. First, ARTL learns a shared classifier for both source and target domain. By contrast, we propose to learn unshared classifiers between the source and target to deal with large shift. Secondly, to cluster the unlabelled target data, we propose to regularize the manifold structure in the target domain only instead of regularizing the cross-domain (i.e. all the data in source and target domains) manifold structure in ARTL. The assumption for doing so in ARTL is that if the marginal distributions of two samples in the source and target domain are similar, the conditional distributions of them are also similar. However, if the geometrical structure of cross-domain data in the original space is preserved, the domain shift cannot be effectively reduced. By contrast, the proposed method does not require this assumption to be true and only regularize the target domain to preserve the structure information in the target domain. Experiments on both synthetic and real-world cross-domain recognition tasks have shown the effectiveness of the proposed method.

2. Proposed method

This section presents the multi-task learning-based Unsupervised Domain Adaptation (mtUDA) method in detail. It begins with the definitions of terminologies. The source domain data denoted as $X_s \in \mathbb{R}^{d\times n_s}$ are draw from distribution $P_s(X_s)$ and the target domain data denoted as $X_t \in \mathbb{R}^{d\times n_t}$ are draw from distribution $P_t(X_t)$, where d is the dimension of the data instance, n_s and n_t are number of samples in source and target domain respectively. We focus on the unsupervised domain adaptation problem which assumes that there are sufficient labelled source domain data, $\mathcal{D}_s = \{(\mathbf{x}_i^s, y_i)\}_{i=1}^{n_s}$, $\mathbf{x}_i^s \in \mathbb{R}^d$, and unlabelled target domain data, $\mathcal{D}_t = \{(\mathbf{x}_j^t)\}_{j=1}^{n_t}$, $\mathbf{x}_j^t \in \mathbb{R}^d$, in the training stage. The feature spaces and label spaces between domains are assumed same: $\mathcal{X}_s = \mathcal{X}_t$ and $\mathcal{Y}_s = \mathcal{Y}_t$. However, due to the dataset shift, $P_s(X_s) \neq P_t(X_t)$ and $P_s(Y_s|X_s) \neq P_t(Y_t|X_t)$. Table 1 lists some notations used in the paper.

2.1. Formulation

2.1.1. Regularized risk minimization

Suppose there is no domain shift between source and target domains, the classifier learnt on the labelled source data can be applied on the target samples directly using the standard regularization based supervised learning algorithm in a Reproducing Kernel Hilbert Space (RKHS),

$$\min_{f\in\mathcal{H}_K} \sum_{i=1}^{n} \mathcal{L}(f(\mathbf{x}_i), y_i) + \gamma \|f\|_K^2 \tag{1}$$

where $\mathcal{H}_K$ is an appropriately chosen RKHS, K is a Mercer kernel, $\mathcal{L}$ denotes some loss functions, which can be squared loss $(y_i - f(\mathbf{x}_i))^2$ for RLS or hinge loss $max[0, 1 - y_i f(\mathbf{x}_i)]$ for SVM, γ is the shrinkage regularization parameter to regularize the complexity of learned model.

However, the source classifier is likely not optimal for the target domain due to the existence of domain shift. Hence, two different classifiers on source and target domain respectively are to learned with additional regularization on the two classifiers in a similar way to multi-task learning.

2.1.2. Multi-task learning

The key idea of multi-task learning (MTL) is that the performances of the related tasks can be boosted by learning them jointly. In general, the MTL is based on the regularized risk minimization learning framework. In the context of domain adaptation, source and target tasks are two related tasks and the MTL formulation can be expressed as follows,

$$\begin{aligned}\min_{f\in\mathcal{H}_K} \frac{1}{n_s}\sum_{i=1}^{n_s} \mathcal{L}(f_s(\mathbf{x}_i^s), y_i) + \frac{1}{n_t}\sum_{j=1}^{n_t} \mathcal{L}(f_t(\mathbf{x}_j^t), y_j) \\ + \gamma_A(\|f_s\|_K^2 + \|f_t\|_K^2) + \gamma_M \Omega(f_s, f_t)\end{aligned} \tag{2}$$

where Ω is a regularization on the source and target classifiers, γ_M is the classifier regularization parameter. In our problem, the source and target tasks are the same, suggesting that the label spaces between domains are identical. A simple regularization is $\Omega(f_s, f_t) = \|f_s - f_t\|_K^2$.

The MTL formulation in Eq. (2) requires labelled data in both domains. However, there are no labelled data in the target domain. Eq. (2) cannot be solved directly. In this paper, it is proposed to leverage the idea of manifold regularization [19] to learn the intrinsic structure of the target domain to treat the target task as an unsupervised clustering task.

2.1.3. Multi-task learning-based unsupervised domain adaptation

Since there are no labelled data in the target domain, the target risk minimization term in Eq. (2) is not computable. This paper proposes to replace the risk minimization term with an intrinsic regularization term to preserve the structure of the target data. Eq. (2) becomes

$$\min_{f\in\mathcal{H}_K}\frac{1}{n_s}\sum_{i=1}^{n_s}\mathcal{L}(f_s(\mathbf{x}_i^s),y_i)+\gamma_I\|f_t\|_I^2+\gamma_A(\|f_s\|_K^2+\|f_t\|_K^2)+\gamma_M\Omega(f_s,f_t) \tag{3}$$

where $\|f_t\|_I^2 = \frac{1}{n_t^2}\sum_{i,j}(f_t(\mathbf{x}_i^t)-f_t(\mathbf{x}_j^t))^2W_{ij} = \frac{1}{n_t^2}tr(\mathbf{f}_t^TL\mathbf{f}_t)$ is the intrinsic manifold regularization for the target domain, $\mathbf{f}_t = [f_t(\mathbf{x}_1^t),\ldots,f_t(\mathbf{x}_{n_t}^t)]^T$, W_{ij} are edge weights in the adjacency graph (with p-nearest neighbours of each data point) of the target data, $L = D - W$ is the graph Laplacian, D is a diagonal matrix given by $D_{ii} = \sum_{i,j=1}^{n_t}W_{ij}$, γ_I is a manifold regularization parameter.

The use of multi-task regularization can remove additional conditions, such as orthogonal constraint, to avoid degenerate solutions as required in manifold regularization-based unsupervised learning [19]. Note that the term $\|f_t\|_I^2$ is different from the manifold regularization term in [13]. It only regularizes the manifold structure in the target domain rather than the cross-domain (i.e. all the data in source and target domains) as done in [13]. The assumption for doing so in [13] is that if the marginal distributions of two samples in the source and target domain are similar, the conditional distributions of them are also similar. However, if the geometrical structure of cross-domain data in the original space is preserved, the domain shift cannot be effectively reduced. By contrast, the proposed method does not require this assumption to be true and treats the source and target domain separately as two joint tasks rather than as a whole for exploiting more domain specific features and reducing the domain shift effectively.

Eq. (3) has considered the source risk minimization, task relatedness of the source and target domains, and target domain intrinsic structure in a multi-task learning framework. The target task here can be seen as an unsupervised clustering task. However, the ultimate goal is to assign class labels to target samples rather than just grouping them. Hence, the class information from the source domain needs to be leveraged. Since there is the shift between the source and target domains and the manifold structure across the domains is not regularized, additional terms to reduce the class distribution shift are required to make sure that the target clusters are assigned with right class labels. This paper proposes to use Maximum Mean Discrepancy (MMD) criterion to regularize both the marginal and the conditional distribution shift [9,13].

The final objective function of the multi-task learning-based Unsupervised Domain Adaptation (mtUDA) method is

$$\begin{aligned}\min_{f_s,f_t\in\mathcal{H}_K}&\frac{1}{n_s}\sum_{i=1}^{n_s}\mathcal{L}(f_s(\mathbf{x}_i^s),y_i)+\gamma_I\|f_t\|_I^2+\gamma_A(\|f_s\|_K^2+\|f_t\|_K^2)\\&+\gamma_M\Omega(f_s,f_t)+\gamma_D D(P_s,P_t)\end{aligned} \tag{4}$$

where γ_D is the MMD regularization parameter,

$$\begin{aligned}D(P_s,P_t)&=\|\frac{1}{n_s}\sum_{i=1}^{n_s}f_s(\mathbf{x}_i^s)-\frac{1}{n_t}\sum_{j=1}^{n_t}f_t(\mathbf{x}_j^t)\|_F^2\\&+\sum_{c=1}^{C}\|\frac{1}{n_s^{(c)}}\sum_{\mathbf{x}_i^s\in X_s^{(c)}}f_s(\mathbf{x}_i^s)-\frac{1}{n_t^{(c)}}\sum_{\mathbf{x}_j^t\in X_t^{(c)}}f_t(\mathbf{x}_j^t)\|_F^2=tr([\mathbf{f}_s^T\quad\mathbf{f}_t^T]M\begin{bmatrix}\mathbf{f}_s\\\mathbf{f}_t\end{bmatrix})\end{aligned} \tag{5}$$

is the MMD measure of joint distribution distance between source and target domains, $M = M_0 + \sum_{c=1}^{C}M_c$, $(M_0)_{ij} = 1/n_s^2$ if $\mathbf{x}_i,\mathbf{x}_j\in\mathcal{D}_s$, $(M_0)_{ij} = 1/n_t^2$ if $\mathbf{x}_i,\mathbf{x}_j\in\mathcal{D}_t$, otherwise $(M_0)_{ij} = -1/n_sn_t$, $(M_c)_{ij} = \frac{1}{n_s^{(c)}n_s^{(c)}}$ if $\mathbf{x}_i,\mathbf{x}_j\in\mathcal{D}_s^{(c)}$, $(M_c)_{ij} = \frac{1}{n_t^{(c)}n_t^{(c)}}$ if $\mathbf{x}_i,\mathbf{x}_j\in\mathcal{D}_t^{(c)}$, $(M_c)_{ij} = \frac{-1}{n_s^{(c)}n_t^{(c)}}$ if $\mathbf{x}_i\in\mathcal{D}_s^{(c)},\mathbf{x}_j\in\mathcal{D}_t^{(c)}$ or $\mathbf{x}_j\in\mathcal{D}_s^{(c)},\mathbf{x}_i\in\mathcal{D}_t^{(c)}$, otherwise $(M_c)_{ij} = 0$, $\mathbf{f}_s = [f_s(\mathbf{x}_1),\ldots,f_s(\mathbf{x}_{n_s})]^T$. Since there is no labelled data in the target domain, pseudo labels are obtained using some base classifiers (e.g. NN) trained on the source domain data in a similar way to [13]. Thus, the conditional distributions can be compared. The pseudo labels are iteratively updated after obtaining the adaptive classifier for the target data. It is worth emphasizing that the source and target classifiers are distinct, which is different from [13].

In the following, two mtUDA algorithms with two different loss functions are presented, namely Regularized Least Squares and Support Vector Machines.

2.2. Regularized least squares algorithm

The Regularized Least Squares algorithm (denoted as mtUDA-RLS) can be expressed as,

$$\begin{aligned}\min_{f_s,f_t\in\mathcal{H}_K}&\frac{1}{n_s}\sum_{i=1}^{n_s}(y_i-f_s(\mathbf{x}_i^s))^2+\frac{\gamma_I}{n_t^2}tr(\mathbf{f}_t^TL\mathbf{f}_t)\\&+\gamma_A(\|f_s\|_K^2+\|f_t\|_K^2)+\gamma_M\|f_s-f_t\|_K^2+\gamma_D tr([\mathbf{f}_s^T\quad\mathbf{f}_t^T]M\begin{bmatrix}\mathbf{f}_s\\\mathbf{f}_t\end{bmatrix})\end{aligned} \tag{6}$$

Based on the Representer Theorem, the solution is an expansion of kernel functions over all the data:

$$f_s^\star(\mathbf{x}^s)=\sum_{i=1}^{n_s+n_t}\alpha_{s_i}^\star K(\mathbf{x}^s,\mathbf{x}_i),f_t^\star(\mathbf{x}^t)=\sum_{i=1}^{n_s+n_t}\alpha_{t_i}^\star K(\mathbf{x}^t,\mathbf{x}_i) \tag{7}$$

Substituting them into Eq. (6), the final objective function can be obtained,

$$\begin{aligned}(\boldsymbol{\alpha}_s^\star,\boldsymbol{\alpha}_t^\star)=\arg\min_{\boldsymbol{\alpha}_s,\boldsymbol{\alpha}_t\in\mathbb{R}^{(n_s+n_t)\times C}}&\frac{1}{n_s}tr((Y_s-\boldsymbol{\alpha}_s^TK_s^T)(Y_s-\boldsymbol{\alpha}_s^TK_s^T)^T)\\&+\frac{\gamma_I}{n_t^2}tr(\boldsymbol{\alpha}_t^TK_t^TLK_t\boldsymbol{\alpha}_t)+\gamma_A tr(\boldsymbol{\alpha}_s^TK\boldsymbol{\alpha}_s+\boldsymbol{\alpha}_t^TK\boldsymbol{\alpha}_t)\\&+\gamma_M tr(\boldsymbol{\alpha}_s^TK\boldsymbol{\alpha}_s-\boldsymbol{\alpha}_s^TK\boldsymbol{\alpha}_t-\boldsymbol{\alpha}_t^TK\boldsymbol{\alpha}_s+\boldsymbol{\alpha}_t^TK\boldsymbol{\alpha}_t)\\&+\gamma_D tr([\boldsymbol{\alpha}_s^TK_s^T\quad\boldsymbol{\alpha}_t^TK_t^T]M\begin{bmatrix}K_s\boldsymbol{\alpha}_s\\K_t\boldsymbol{\alpha}_t\end{bmatrix})\end{aligned} \tag{8}$$

where $Y_s\in\mathbb{R}^{C\times n_s}$ is a label matrix, $y_s^c = 1$ if $y_s(x) = c$, otherwise $y_s^c = 0$ (mtUDA-RLS can be naturally applied to multi-class classification problem directly with this form of label matrix), $K = \Phi(X)^T\Phi(X)$, $K_s = \Phi(X_s)^T\Phi(X)$, and $K_t = \Phi(X_t)^T\Phi(X)$ are the kernel matrices, $X = [X_s, X_t]$ denotes all the source and target training samples, $\Phi(X) = [\phi(x_1),\ldots,\phi(x_{n_s+n_t})]$ is the feature mappings to a space of a higher or even infinite dimension. To simultaneously optimize $\boldsymbol{\alpha}_s$ and $\boldsymbol{\alpha}_t$, we write $[\boldsymbol{\alpha}_s^T\quad\boldsymbol{\alpha}_t^T]$ as $\boldsymbol{\alpha}^T$. Setting the derivatives with respect to $\boldsymbol{\alpha}$ to zero, the closed form solution $\boldsymbol{\alpha}^\star$ is obtained

$$\begin{aligned}\boldsymbol{\alpha}^\star=\Bigg(&\begin{bmatrix}K_s^TK_s&0\\0&0\end{bmatrix}+\frac{\gamma_In_s}{n_t^2}\begin{bmatrix}0&0\\0&K_t^TLK_t\end{bmatrix}+\gamma_An_s\begin{bmatrix}K&0\\0&K\end{bmatrix}\\&+\gamma_Mn_s\begin{bmatrix}K&-K\\-K&K\end{bmatrix}+\gamma_Dn_s\begin{bmatrix}K_s^T&0\\0&K_t^T\end{bmatrix}M\begin{bmatrix}K_s^T&0\\0&K_t^T\end{bmatrix}^T+\epsilon I\Bigg)^{-1}\begin{bmatrix}K_s^TY_s\\0\end{bmatrix}\end{aligned} \tag{9}$$

where $Y_s\in\mathbb{R}^{C\times n_s}$ is the one-hot label matrix, $K = \Phi(X)^T\Phi(X)$, $K_s = \Phi(X_s)^T\Phi(X)$, and $K_t = \Phi(X_t)^T\Phi(X)$ are the kernel matrices, $\Phi(X) = [\phi(x_1),\ldots,\phi(x_{n_s+n_t})]$ is the feature mappings to a space

Table 2
Accuracy on the digit dataset.

Datasets	MNIST→USPS	USPS→MNIST	Avg.
No Adaptation	65.9	44.7	55.3
SA [10]	67.8	48.8	58.3
JDA [9]	67.3	59.7	63.5
JGSA [12]	80.4	68.2	74.3
ARSVMlinear [13]	67.8	57.2	62.5
ARSVMGaussian [13]	88.2	64.4	76.3
ARRLSlinear [13]	67.2	52.1	59.6
ARRLSGaussian [13]	88.8	67.7	78.2
mtUDA-SVMlinear	75.7	62.6	69.2
mtUDA-SVMGaussian	88.3	68.0	78.1
mtUDA-RLSlinear	70.3	51.4	60.8
mtUDA-RLSGaussian	**89.2**	**71.9**	**80.5**

of a higher or even infinite dimension, ϵI is included to obtain numerically more stable solution. Following [19], each trade-off coefficient is treated as a whole, e.g. $\hat{\gamma_I} = \frac{\gamma_I n_s}{n_t^2}$, $\hat{\gamma_A} = \gamma_A n_s$, $\hat{\gamma_M} = \gamma_M n_s$, and $\hat{\gamma_D} = \gamma_D n_s$, when tuning the parameters.

2.3. Support vector machines algorithm

The Support Vector Machines algorithm (denoted as mtUDA-SVM) can be formulated as,

$$\begin{aligned}\min_{f_s, f_t \in \mathcal{H}_K} & \frac{1}{n_s}\sum_{i=1}^{n_s}\max(0, 1 - y_i f_s(\mathbf{x}_i))_+ \frac{\gamma_I}{n_t^2} tr(\mathbf{f}_t^T L \mathbf{f}_t) \\ & + \gamma_A(\|f_s\|_K^2 + \|f_t\|_K^2) + \gamma_M \|f_s - f_t\|_K^2 + \gamma_D tr([\mathbf{f}_s^T \quad \mathbf{f}_t^T] M \begin{bmatrix}\mathbf{f}_s \\ \mathbf{f}_t\end{bmatrix})\end{aligned} \tag{10}$$

Based on the Representer Theorem, substituting Eqs. (7) into Eq. (10), and following [19] to solve the objective function by introducing the Lagrangian, yields

$$\beta^\star = \max_{\beta \in \mathbb{R}^{n_s}} \sum_{i=1}^{n_s} \beta_i - \frac{1}{2}\beta^T Q \beta \tag{11}$$

$$s.t. \quad \sum_{i=1}^{n_s} \beta_i y_i = 0, \quad 0 \le \beta_i \le \frac{1}{n_s}, \quad i = 1, \ldots, n_s$$

where

$$\begin{aligned} Q = & \begin{bmatrix} K_s^T \tilde{Y}_s \\ 0 \end{bmatrix}^T \left(\frac{2\gamma_I}{n_t^2} \begin{bmatrix} 0 & 0 \\ 0 & K_t^T L K_t \end{bmatrix} + 2\gamma_A \begin{bmatrix} K & 0 \\ 0 & K \end{bmatrix} \right. \\ & \left. + 2\gamma_M \begin{bmatrix} K & -K \\ -K & K \end{bmatrix} + 2\gamma_D \begin{bmatrix} K_s^T & 0 \\ 0 & K_t^T \end{bmatrix} M \begin{bmatrix} K_s^T & 0 \\ 0 & K_t^T \end{bmatrix}^T + \epsilon I \right)^{-1} \\ & \times \begin{bmatrix} K_s^T \tilde{Y}_s \\ 0 \end{bmatrix}, \end{aligned}$$

and $\tilde{Y}_s = diag(y_1, \ldots, y_{n_s})$. Then a standard SVM solver [20] with quadratic form induced by Q can be used to obtain $\beta^\star$, and the final classifier can be obtained by

$$\begin{aligned} \alpha^\star = & \left(\frac{2\gamma_I}{n_t^2} \begin{bmatrix} 0 & 0 \\ 0 & K_t^T L K_t \end{bmatrix} + 2\gamma_A \begin{bmatrix} K & 0 \\ 0 & K \end{bmatrix} + 2\gamma_M \begin{bmatrix} K & -K \\ -K & K \end{bmatrix} \right. \\ & \left. + 2\gamma_D \begin{bmatrix} K_s^T & 0 \\ 0 & K_t^T \end{bmatrix} M \begin{bmatrix} K_s^T & 0 \\ 0 & K_t^T \end{bmatrix}^T + \epsilon I \right)^{-1} \begin{bmatrix} K_s^T \tilde{Y}_s \\ 0 \end{bmatrix} \beta^\star \end{aligned} \tag{12}$$

where ϵI is also added to obtain numerically stable solution. For multi-class classification problem, one-against-all strategy using mtUDA-SVM method is employed. Similar to mtUDA-RLS and [19], each trade-off coefficient is treated as a whole, e.g. $\hat{\gamma_I} = \frac{\gamma_I}{n_t^2}$, $\hat{\gamma_A} = \gamma_A$, $\hat{\gamma_M} = \gamma_M$, and $\hat{\gamma_D} = \gamma_D$. For easily tuning the parameters, M and L are normalized.

Table 3
Accuracy on the Office-31 dataset.

Datasets	A→W	A→D	W→A	W→D	D→A	D→W	Avg.
No Adaptation	55.6	59.2	41.8	98.2	44.9	93.1	65.5
SA [10]	55.4	58.4	44.0	98.6	46.5	92.7	65.9
JDA [9]	56.9	56.6	45.0	98.0	47.0	94.1	66.3
JGSA [12]	60.4	65.3	54.0	97.2	51.5	96.4	70.8
ARSVMlinear [13]	63.4	61.5	50.8	90.8	55.9	91.7	69.0
ARSVMGaussian [13]	65.5	66.9	57.1	92.4	60.7	93.5	72.7
ARRLSlinear [13]	66.0	69.7	57.4	97.8	58.5	94.5	74.0
ARRLSGaussian [13]	68.3	70.3	**60.2**	98.4	61.5	95.2	75.6
DAN [16]	68.5	67.0	53.1	99.0	54.0	96.0	72.9
RTN [17]	70.2	69.3	51.0	**99.3**	50.5	**96.8**	72.9
WDAN [18]	66.8	64.5	52.7	98.7	53.8	95.9	72.1
DANN [15]	**73.0**	72.3	51.2	99.2	53.4	96.4	74.3
mtUDA-SVMlinear	67.4	65.9	54.6	88.6	59.9	88.8	70.9
mtUDA-SVMGaussian	68.1	66.7	57.0	90.6	62.2	92.8	72.9
mtUDA-RLSlinear	69.2	72.9	58.0	98.4	61.4	95.4	75.9
mtUDA-RLSGaussian	70.4	**73.1**	58.8	98.4	**63.1**	96.0	**76.6**

3. Experiments

3.1. Set-ups

Datasets: The proposed mtUDA methods were evaluated on a synthetic dataset, real-world digit recognition datasets and object recognition datasets. The synthetic data has two classes. The two classes of source data are generated by sampling from Gaussians centred at (0,2), and (2,0), while the two classes of target data are sampled from Gaussians centred at (-1,-1), and (2,0).

For cross-domain hand-written digit recognition, MNIST [21] and USPS [22] datasets were used. Data released by [9,13] were used to construct a pair of cross-domain datasets USPS v.s. MNIST.

For object recognition, the methods were evaluated on two different datasets. One is Office-31 dataset studied by Saenko et al. [23], which contains three different domains: Amazon, Webcam, and DSLR. There are 31 classes of objects shared by the three domains, forming 6 pairs of cross-domain tasks. *Decaf*$_7$ features [24] (extracted from AlexNet [25])[1] were used. The other object dataset is the Office-Caltech-10 dataset released by Gong et al. [4]. This dataset is built upon the Office-31 dataset and contains images from four different domains: Amazon, Webcam, DSLR, and Caltech-256 [26]. Ten classes common to four datasets are selected to form 12 pairs of datasets. Two types of features are considered: *Decaf*$_7$ features [24] (extracted from AlexNet [25]) and SURF descriptors. The proposed model is applicable to any features, hand-crafted ones or deeply learned ones with different DNN models such as ResNet [27], GoogleNet [28], and DenseNet [29]. We choose to use the features extracted from AlexNet for fair comparison to the existing domain adaptation methods.

Baselines and Settings: The proposed methods are compared with the state-of-the-art unsupervised domain adaptation method, namely SA [10], DIP [11], JDA [9], JGSA [12], ARTL [13] (includes ARRLS and ARSVM), DANN [15], DAN [16], RTN [17], and WDAN [18]. SA, DIP, JDA, and JGSA are feature transformation-based methods. They transforms the original feature into another space where the distributions of the two domains would be similar. ARTL is a classifier-based method, which aims at learning a single and shared classifier directly such that the distribution of classification scores on the source data aligns with that on the target data. Differently, DANN, DAN, RTN, and WDAN are deep learning based methods, which extend the deep CNNs to domain adaptation by adding some mechanisms (e.g. MMD criterion in DAN, RTN, and WDAN, or domain-adversarial training

[1] https://cs.stanford.edu/~jhoffman/domainadapt/.

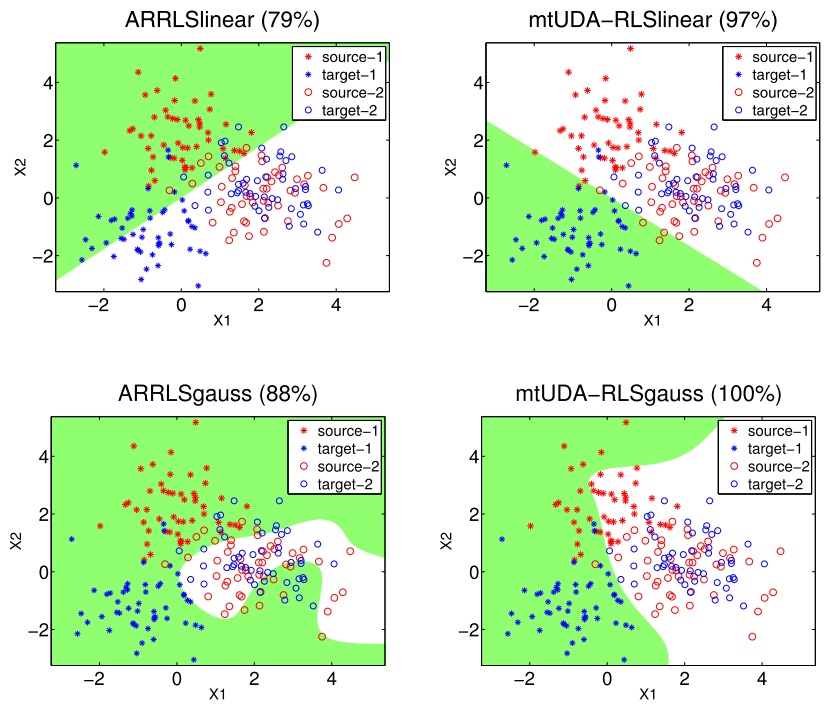

Fig. 1. Comparisons of ARRLS [13] and the proposed mtUDA-RLS on the synthetic data. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 4
Accuracy on the Office-Caltech-10 dataset with deep features.

Datasets	C→A	C→W	C→D	A→C	A→W	A→D	W→C	W→A	W→D	D→C	D→A	D→W	Avg.
No Adaptation	90.5	77.6	82.2	83.6	74.2	86.0	77.4	80.2	**100.0**	79.5	86.9	99.7	84.8
SA [10]	90.0	85.8	86.0	80.0	86.4	88.5	79.6	85.2	99.4	77.9	87.5	95.3	86.8
JDA [9]	89.6	78.6	87.3	81.8	77.3	83.4	83.1	90.4	**100.0**	82.6	91.7	98.3	87.0
JGSA [12]	90.9	86.8	89.8	85.6	84.1	88.5	87.4	92.0	**100.0**	86.6	92.8	99.0	90.3
ARSVMlinear [13]	92.7	94.9	93.6	88.4	91.2	89.2	87.2	92.8	99.4	86.9	**93.7**	96.3	92.2
ARSVMGaussian [13]	92.6	**95.6**	95.6	88.3	90.9	89.2	87.2	92.8	99.4	87.0	93.2	96.3	92.3
ARRLSlinear [13]	92.9	95.3	93.0	88.1	90.2	89.8	87.1	92.7	99.4	87.7	93.6	99.0	92.4
ARRLSGaussian [13]	92.9	94.9	93.6	87.9	90.5	89.8	87.0	92.7	99.4	87.5	93.5	99.0	92.4
WDAN (AlexNet) [18]	93.1	93.6	93.4	86.9	–	–	84.1	–	–	83.9	–	–	89.2
DANN [15]	**93.3**	94.2	89.2	86.6	**91.9**	90.5	87.5	**93.8**	99.4	86.3	93.3	98.6	92.1
mtUDA-SVMlinear	93.1	92.2	**98.1**	**89.1**	91.2	**93.6**	88.1	92.6	98.1	**89.1**	**93.7**	99.7	**93.2**
mtUDA-SVMGaussian	93.0	91.9	97.5	88.3	91.5	93.0	88.1	92.5	97.5	88.8	**93.7**	99.7	92.9
mtUDA-RLSlinear	93.0	91.5	96.8	88.6	91.2	**93.6**	88.6	92.6	98.7	88.7	93.8	**100.0**	93.1
mtUDA-RLSGaussian	92.9	91.5	96.8	88.6	91.2	**93.6**	**89.0**	92.3	98.7	88.6	93.7	**100.0**	93.1

in DANN). These methods are very competitive especially the DANN method, which employs the idea of adversarial training to learn the domain invariant deep features by confusing the domain discriminator. For the shallow learning-based baseline methods, we use the same feature and settings as the proposed method. For deep learning based methods, AlexNet [25] is used as the base model for fair comparison. The proposed methods are also compared with the No Adaptation baseline, which is the results obtained by the Nearest Neighbour classifier on the source domain data without adaptation. For all the compared methods, the parameters recommended by the original papers were used. For the proposed methods, $\hat{\gamma_M} = 1$, $\hat{\gamma_A} = 0.1$, $\hat{\gamma_I} = 1$, $p = 5$, and (1) $\hat{\gamma_D} = 10$ for Office-31; (2) $\hat{\gamma_D} = 1$ for other datasets. In fact, the proposed methods perform well on a wide range of parameter values based on the empirical results. The number of iterations is fixed to 10 since the algorithms generally converge within 10 iterations. All algorithms were evaluated in a fully transductive setup [30].

3.2. Results

Fig. 1 shows the comparison between ARRLS [13] and the proposed mtUDA-RLS methods with linear and Gaussian kernels. The green shade represents the adaptive classifier. The ARRLS method tries to learn a unified classifier that can perform well in both domains, which does not exist or is hard to find when the domain shift is large. Since the goal of the adaptive classifier is to classify the target samples, it is not necessary to perform well in both domains. By contrast, our mtUDA-RLS method jointly learns two different but related classifiers for source and target domain respectively. Without the unified classifier constraint, the target domain adaptive classifier obtained by our mtUDA-RLS method performs much better than ARRLS on the target task.

For the real world datasets, the comparison results in Tables 2–5 show that the proposed method outperforms the state-of-the-art domain adaptation methods on most of the datasets. Based on the results, it is observed that all the domain adaptation methods outperform the No Adaptation results, which means that the domain shift indeed exists on these datasets and both feature

Table 5
Accuracy on the Office-Caltech-10 dataset with SURF features.

Datasets	C→A	C→W	C→D	A→C	A→W	A→D	W→C	W→A	W→D	D→C	D→A	D→W	Avg.
No Adaptation	36.0	29.2	38.2	34.2	31.2	35.7	28.8	31.6	84.7	29.6	28.3	83.7	40.9
SA [10]	49.3	40.0	39.5	40.0	33.2	33.8	35.2	39.3	75.2	**34.6**	39.9	77.0	44.7
JDA [9]	44.8	41.7	45.2	39.4	38.0	39.5	31.2	32.8	89.2	31.5	33.1	89.5	46.3
DIP [11]	51.9	49.5	**50.3**	39.3	39.7	45.8	**38.4**	**42.8**	70.7	33.8	**40.1**	73.2	48.0
JGSA [12]	51.5	45.4	45.9	**41.5**	45.8	47.1	33.2	39.9	90.5	29.9	38.0	**91.9**	50.0
ARRLSlinear [13]	53.4	43.4	40.8	38.8	38.0	38.2	32.3	38.4	88.5	31.6	36.3	89.2	47.4
ARRLSGaussian [13]	50.7	43.1	46.5	40.9	40.7	42.7	31.5	38.7	**91.1**	30.5	32.6	**91.9**	48.4
mtUDA-RLSlinear	51.7	47.8	42.7	38.5	44.4	47.8	31.8	40.6	84.7	30.0	37.1	90.9	49.0
mtUDA-RLSGaussian	**55.1**	**56.3**	**50.3**	40.0	**50.2**	**49.0**	32.8	40.6	86.0	30.5	35.5	90.9	**51.4**

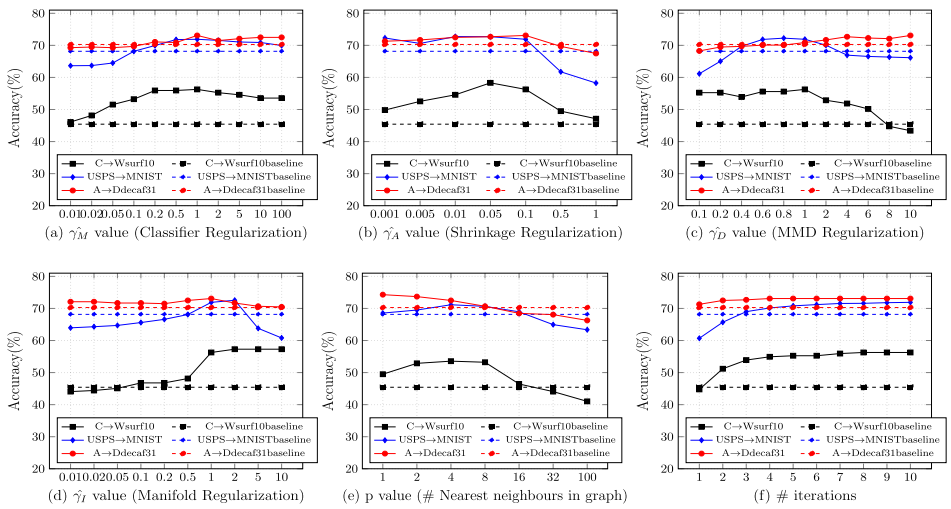

Fig. 2. Parameter sensitivity of the proposed method.

transformation-based methods and classifier-based methods can reduce the shift to different degrees. Secondly, the classifier-based methods perform better than feature transformation-based methods in general on the evaluated datasets, which verifies that the two-step solution in the feature transformation-based methods may not be optimal. This is because these methods may fail to consider the discriminative information when aligning distributions in the feature space. Thirdly, the proposed mtUDA methods outperform both ARTL and the deep-learning-based methods (i.e. DANN, DAN, RTN, WDAN). As analysed before, ARTL assumes shared classifier between domains, which may not exist. In addition, compared to ARTL, the good results obtained by mtUDA are also due to the application of manifold regularization to target data only. Similarly, compared to the competitive deep-learning-based methods, better performance can be obtained with much lower computational cost. This may be because most of these methods still follow the idea of shared classifiers (parameters), which is limited when the domain shift is large. By contrast, the learned unshared classifiers in the proposed method consider more domain specific discriminative information. Lastly, the comparison between the linear and Gaussian kernel versions of the proposed algorithms have shown that, for the digit recognition tasks, the Gaussian kernel outperforms linear kernel to a large degree, but the differences are not obvious on the object recognition tasks. In general, the Gaussian kernel versions of the proposed algorithms can obtain the state-of-the-art results.

Here the computational complexity of the proposed mtUDA method is analysed. Let the average number of non-zero features per example be s. Firstly, for solving the linear systems (Eqs. (8) and (11)) using LU decomposition requires $O((2(n_s+n_t))^3)$. Secondly, constructing the graph Laplacian matrix L for the target domain needs $O(s(n_t)^2)$. Thirdly, constructing the kernel matrix K and aggregating MMD matrix M require $O(C(n_s+n_t)^2)$. Hence, the computational complexity of the proposed method is $O((2(n_s+n_t))^3)+O(s(n_t)^2)+O(C(n_s+n_t)^2)$. Conjugate gradient methods can be easily used to speed up the algorithms, which is part of our future work. We take the Office-Caltech-10 with SURF features as an example to compare the CPU time of the proposed method with previous methods. The CPU time (training and evaluation) of mtUDA-SVM is 26.9s and mtUDA-RLS is 111.7s on average over different domain pairs. Compared to the previous state-of-the-art shallow-learning-based method, ARRLS (2.43s), the average runtime of mtUDA is longer, but is much shorter than DIP (728.7s) and similar to JGSA (28.9s), however, better performance can be obtained on most of the domain pairs. Compared to the deep-learning-based methods, the runtime is much shorter and the performance is similar or better.

3.3. Parameter sensitivity

Experiments were conducted on the C→W (Office-Caltech-10 dataset with SURF descriptor), USPS→MNIST, and A→D (Office-31 dataset with $decaf_7$ feature) datasets to study the sensitivity of the proposed algorithms to their parameters (Fig. 2). The solid lines are the accuracies obtained by mtUDA-RLS with Gaussian kernel, and the dashed lines are the results obtained by the best baseline methods on each dataset. It can be seen that a wide range of values can be chosen to obtain satisfactory performances. $\hat{\gamma_M}$ regularizes the similarity between source and target classifiers. If $\hat{\gamma_M}$ is too small, the source class information cannot be transferred to the target domain, if $\hat{\gamma_M} \rightarrow \infty$ the source and target classifiers tend to be the same, which may not be desirable. $\hat{\gamma_A}$ controls the complexity of the classifiers. A small $\hat{\gamma_A}$ would lead to overfitting while a too large $\hat{\gamma_A}$ leads that the models cannot fit the data. $\hat{\gamma_D}$ controls the degree of distribution shift. Though the larger $\hat{\gamma_D}$ will lead to smaller distribution shift, a too

large value will cancel out other regularizations. $\hat{\gamma_l}$ regularizes the geometry structure of the target domain. If $\hat{\gamma_l}$ is too small, the target domain structure is not preserved, but if $\hat{\gamma_l}$ is too large the source class information is discarded. The number p of nearest neighbours in constructing the adjacency graph determines the degree of target structure preservation. A too small p captures little structure information, while a too large p connects samples without similarity. For the number of iterations, the results show that the accuracies increase steadily with more iterations and converge to the best value within 10 iterations. Though there are 4 hyper parameters for the algorithm, a wide range of values can be chosen. Specifically, $\hat{\gamma_M} \in [0.1, 100]$, $\hat{\gamma_A} \in [0.001, 0.1]$, $\hat{\gamma_A} \in [0.4, 2]$, $\hat{\gamma_A} \in [0.5, 2]$ can be optimal parameter values. In addition, some of them can be fixed to obtain good results consistently for different datasets. For example, in our experiments, $\hat{\gamma_M} = 1$, $\hat{\gamma_A} = 0.1$, $\hat{\gamma_l} = 1$ are used for all the datasets.

4. Conclusion

This paper presents a novel multi-task learning-based unsupervised domain adaptation method. It relaxes the single classifier assumption in the conventional classifier-based unsupervised domain adaptation and proposes to jointly optimize source and target classifiers by considering the manifold structure of the target domain and the distribution divergence between the domains. Experimental results on both synthetic and real-world cross domain recognition datasets have shown the effectiveness of the proposed method.

References

[1] J. Yang, R. Yan, A.G. Hauptmann, Cross-domain video concept detection using adaptive svms, in: Proc. ACM International Conference on Multimedia, ACM, 2007, pp. 188–197.

[2] B. Quanz, J. Huan, Large margin transductive transfer learning, in: Proc. ACM Conference on Information and Knowledge Management, ACM, 2009, pp. 1327–1336.

[3] S.J. Pan, I.W. Tsang, J.T. Kwok, Q. Yang, Domain adaptation via transfer component analysis, IEEE Trans. Neural Netw. 22 (2) (2011) 199–210.

[4] B. Gong, Y. Shi, F. Sha, K. Grauman, Geodesic flow kernel for unsupervised domain adaptation, in: Proc. IEEE Conference on Computer Vision and Pattern Recognition, IEEE, 2012, pp. 2066–2073.

[5] J. Lu, V. Behbood, P. Hao, H. Zuo, S. Xue, G. Zhang, Transfer learning using computational intelligence: a survey, Knowl.-Based Syst. 80 (2015) 14–23.

[6] L. Duan, D. Xu, I.W. Tsang, Learning with augmented features for heterogeneous domain adaptation, in: Proc. International Conference on Machine Learning, 2012, pp. 711–718.

[7] W. Li, L. Duan, D. Xu, I.W. Tsang, Learning with augmented features for supervised and semi-supervised heterogeneous domain adaptation, IEEE Trans. Pattern Anal. Mach. Intell. 36 (6) (2014) 1134–1148.

[8] F. Liu, J. Lu, G. Zhang, Unsupervised heterogeneous domain adaptation via shared fuzzy equivalence relations, IEEE Trans. Fuzzy Syst. 26 (6) (2018) 3555–3568.

[9] M. Long, J. Wang, G. Ding, J. Sun, P. Yu, Transfer feature learning with joint distribution adaptation, in: Proc. IEEE International Conference on Computer Vision, IEEE, 2013, pp. 2200–2207.

[10] B. Fernando, A. Habrard, M. Sebban, T. Tuytelaars, Unsupervised visual domain adaptation using subspace alignment, in: Proc. IEEE International Conference on Computer Vision, 2013, pp. 2960–2967.

[11] M. Baktashmotlagh, M.T. Harandi, B.C. Lovell, M. Salzmann, Unsupervised domain adaptation by domain invariant projection, in: Proc. IEEE International Conference on Computer Vision, IEEE, 2013, pp. 769–776.

[12] J. Zhang, W. Li, P. Ogunbona, Joint geometrical and statistical alignment for visual domain adaptation, in: Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 1859–1867.

[13] M. Long, J. Wang, G. Ding, S.J. Pan, P.S. Yu, Adaptation regularization: A general framework for transfer learning, IEEE Trans. Knowl. Data Eng. 26 (5) (2014) 1076–1089.

[14] L. Duan, D. Xu, I.-H. Tsang, J. Luo, Visual event recognition in videos by learning from web data, IEEE Trans. Pattern Anal. Mach. Intell. 34 (9) (2012) 1667–1680.

[15] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, V. Lempitsky, Domain-adversarial training of neural networks, J. Mach. Learn. Res. 17 (59) (2016) 1–35.

[16] M. Long, J. Wang, Learning transferable features with deep adaptation networks, in: Proc. International Conference on Machine Learning, 2015, pp. 97–105.

[17] M. Long, H. Zhu, J. Wang, M.I. Jordan, Unsupervised domain adaptation with residual transfer networks, in: Advances in Neural Information Processing Systems, 2016, pp. 136–144.

[18] H. Yan, Y. Ding, P. Li, Q. Wang, Y. Xu, W. Zuo, Mind the class weight bias: weighted maximum mean discrepancy for unsupervised domain adaptation, in: Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 2272–2281.

[19] M. Belkin, P. Niyogi, V. Sindhwani, Manifold regularization: A geometric framework for learning from labeled and unlabeled examples, J. Mach. Learn. Res. 7 (Nov) (2006) 2399–2434.

[20] C.-C. Chang, C.-J. Lin, LIBSVM: a library for support vector machines, ACM Trans. Intell. Syst. Technol. (TIST) 2 (3) (2011) 27.

[21] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86 (11) (1998) 2278–2324.

[22] J.J. Hull, A database for handwritten text recognition research, IEEE Trans. Pattern Anal. Mach. Intell. 16 (5) (1994) 550–554.

[23] K. Saenko, B. Kulis, M. Fritz, T. Darrell, Adapting visual category models to new domains, in: Proc. European Conference on Computer Vision, Springer, 2010, pp. 213–226.

[24] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, T. Darrell, DeCAF: A deep convolutional activation feature for generic visual recognition, in: Proc. International Conference on Machine Learning, 2014, pp. 647–655.

[25] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, in: Advances in Neural Information Processing Systems, 2012, pp. 1097–1105.

[26] G. Griffin, A. Holub, P. Perona, Caltech-256 object category dataset, Tech. rep., California Institute of Technology, 2007.

[27] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proc. Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.

[28] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 1–9.

[29] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 4700–4708.

[30] B. Gong, K. Grauman, F. Sha, Connecting the dots with landmarks: Discriminatively learning domain-invariant features for unsupervised domain adaptation, in: Proc. International Conference on Machine Learning, 2013, pp. 222–230.